Deliverable 4

AI feature 1: Intelligent Appointment Prioritization & Slot Recommendation

Overview:

This function goal is to take the patient health information and based on that the function should decide how urgent the case is by providing an urgency score, and priority level. After that, based on the doctor's availability and preferred time the function shall recommend the best available slot.

This will directly support reducing waiting time by 50% for the patients as stated in the project objectives.

Processing logic:

patient data > calculating urgency > prioritize patient> doctor availability> recommend slot

System Prompt:

You are a healthcare AI assistant your goal is prioritizing patient appointments and recommend the most appropriate time slot.

Analyze patient health information and based on that calculate the urgency score of that case. Given these factors: age, symptoms, pain level, condition type, and vitals, calculate an urgency score from 0% to 100%.

Based on urgency score, classify the patient into (low, medium, high) priority. After that, using available doctor slots then recommend the most appropriate appointment slot.

Use few-shot learning technique:

Example 1:

Input:
Age: 23
Symptoms: mild sore throat
Pain level: 2
Condition: general
Vitals: normal
Preferred date: tomorrow
Preferred time: afternoon
Available slots: Tomorrow 1:00 PM, Tomorrow 3:00 PM

Output:
{
"urgency_score": 20,
"priority_level": "Low",
"recommended_slot": "Tomorrow 1:00 PM",
"reason": "Mild symptoms and normal vitals. Patient preference considered."
}

Example 2:

Input:
Age: 67
Symptoms: severe headache, blurred vision
Pain level: 8
Condition: neurological
Vitals: blood pressure high
Preferred date: next week
Preferred time: morning
Available slots: Today 6:00 PM, Tomorrow 8:00 AM

Output:
{
"urgency_score": 87,
"priority_level": "High",
"recommended_slot": "Today 6:00 PM",
"reason": "Urgent symptoms and abnormal vitals require earlier appointment."
}

Input:

Age: 52
Symptoms: chest pain, shortness of breath, dizziness
Pain level: 9
Condition: cardiac
Vitals: blood pressure high, heart rate high
Preferred date: tomorrow
Preferred time: morning
Available slots: Today 2:00 PM - Dr. Ahmed, today 4:30 PM - Dr. Sara, Tomorrow 9:00 AM - Dr. Ahmed

Output:

{

"urgency_score": 94,

"priority_level": "High",

"recommended_slot": "Today 2:00 PM - Dr. Ahmed",

"reason": "Severe cardiac-related symptoms, high pain level, and abnormal vitals indicate a high-priority case. The earliest available slot was recommended."

}

Python Code using Google Collab:

```python
from __future__ import annotations
import re
import json
from typing import List, Dict, Any, Tuple
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MultiLabelBinarizer, OneHotEncoder
# 1) Custom transformers for ML input
1 usage
class SymptomsEncoder(BaseEstimator, TransformerMixin):
    def __init__(self):
        self.mlb = MultiLabelBinarizer()
    2 usages (2 dynamic)
    def fit(self, X, y=None):
        cleaned = [self._clean_list(row) for row in X]
        self.mlb.fit(cleaned)
        return self
    1 usage (1 dynamic)
    def transform(self, X):
        cleaned = [self._clean_list(row) for row in X]
        return self.mlb.transform(cleaned)
    2 usages
    def _clean_list(self, symptoms):
        if isinstance(symptoms, list):
            return [str(s).strip().lower() for s in symptoms]
        return []
1 usage
class VitalsEncoder(BaseEstimator, TransformerMixin):
    2 usages (2 dynamic)
    def fit(self, X, y=None):
        return self
    1 usage (1 dynamic)
    def transform(self, X):
        rows = []
        for vitals in X:
            vitals = vitals if isinstance(vitals, dict) else {}
            v = {k.lower(): str(val).lower() for k, val in vitals.items()}

            row = [
                1 if v.get("blood_pressure") in ["high", "low", "abnormal"] else 0,
                1 if v.get("temperature") in ["high", "low", "fever", "abnormal"] else 0,
                1 if v.get("heart_rate") in ["high", "low", "abnormal", "tachycardia", "bradycardia"] else 0,
                1 if v.get("oxygen") in ["low", "abnormal"] else 0,
                1 if v.get("general") in ["abnormal", "unstable"] else 0,
            ]
            rows.append(row)

        return np.array(rows)

```

```python
def train_model():
    X_train, y_train = build_training_data()
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", "passthrough", ["age", "pain_level"]),
            ("condition", OneHotEncoder(handle_unknown="ignore"), ["condition_type"]),
            ("symptoms", SymptomsEncoder(), "symptoms"),
            ("vitals", VitalsEncoder(), "vitals"),
        ]
    )
    model = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("regressor", RandomForestRegressor(n_estimators=200, random_state=42))
    ])
    model.fit(X_train, y_train)
    return model
# 3) Required prototype functions
1 usage
def calculate_urgency_score(
    model,
    age: int,
    symptoms: List[str],
    pain_level: int,
    condition_type: str,
    vitals: Dict[str, str]
) -> int:
    """
    Uses the machine learning model to predict urgency score from 0 to 100.
    """
    patient_input = [{
        "age": age,
        "symptoms": symptoms,
        "pain_level": pain_level,
        "condition_type": condition_type,
        "vitals": vitals
    }]
    score = model.predict(patient_input)[0]
    # Safety adjustment for prototype
    symptom_text = [s.lower() for s in symptoms]
    abnormal_vitals_count = sum(
        1 for value in vitals.values()
        if str(value).lower() in ["high", "low", "abnormal", "fever", "unstable"]
    )
    if "chest pain" in symptom_text or "shortness of breath" in symptom_text:
        score += 10
    if "blurred vision" in symptom_text or "severe headache" in symptom_text:
        score += 8
    if age >= 65:
        score += 5
    score += abnormal_vitals_count * 3
    score = max(0, min(100, round(score)))
    return score

```

```python
1 usage
def classify_priority(urgency_score: int) -> str:
    """
    Classifies urgency score into Low, Medium, or High priority.
    """
    if urgency_score >= 75:
        return "High"
    elif urgency_score >= 40:
        return "Medium"
    else:
        return "Low"

1 usage
def parse_slot(slot: str) -> Dict[str, Any]:
    """
    Example:
    'Tomorrow 1:00 PM - Dr. Sara'
    'Today 6:00 PM'
    """
    pattern = r"^(Today|Tomorrow)\s+(\d{1,2}:\d{2}\s*[AP]M)(?:\s*-\s*(.+))?$"
    match = re.match(pattern, slot.strip(), re.IGNORECASE)
    if not match:
        return {
            "raw": slot,
            "day": "Unknown",
            "time": "Unknown",
            "doctor": None
        }
    return {
        "raw": slot,
        "day": match.group(1).title(),
        "time": match.group(2).upper(),
        "doctor": match.group(3).strip() if match.group(3) else None
    }

2 usages
def check_doctor_availability(available_slots: List[str]) -> List[Dict[str, Any]]:
    """
    Returns all available slots in structured format.
    """
    return [parse_slot(slot) for slot in available_slots]

1 usage
def time_to_number(time_str: str) -> int:
    """
    Converts '1:00 PM' to 1300 for sorting.
    """
    time_str = time_str.strip().upper()
    match = re.match( pattern: r"(\d{1,2}):(\d{2})\s*(AM|PM)", time_str)
    if not match:
        return 9999
    hour = int(match.group(1))
    minute = int(match.group(2))
    period = match.group(3)
    if period == "PM" and hour != 12:
        hour += 12
    if period == "AM" and hour == 12:
        hour = 0
    return hour * 100 + minute

1 usage
```

```python
def recommend_slot(
    priority_level: str,
    preferred_date: str,
    preferred_time: str,
    available_slots: List[str]
) -> str:
    """
    Recommends the best appointment slot.
    """
    slots = check_doctor_availability(available_slots)
    if not slots:
        return "No available slots"
    preferred_date = preferred_date.lower().strip()
    preferred_time = preferred_time.lower().strip()
    def preference_score(slot):
        score = 0
        # Date preference
        if preferred_date == "today":
            if slot["day"].lower() == "today":
                score -= 20
        elif preferred_date == "tomorrow":
            if slot["day"].lower() == "tomorrow":
                score -= 20
        elif preferred_date == "next week":
            score += 10
        # Time preference
        slot_time_num = time_to_number(slot["time"])
        if preferred_time == "morning" and 600 <= slot_time_num < 1200:
            score -= 10
        elif preferred_time == "afternoon" and 1200 <= slot_time_num < 1700:
            score -= 10
        elif preferred_time == "evening" and 1700 <= slot_time_num <= 2100:
            score -= 10
        # High priority -> earliest appointment
        if priority_level == "High":
            if slot["day"].lower() == "today":
                score -= 30
            score += slot_time_num / 1000
        # Medium priority -> moderate balance
        elif priority_level == "Medium":
            if slot["day"].lower() == "today":
                score -= 10
            score += slot_time_num / 2000
        # Low priority -> preference matters more
        else:
            score += slot_time_num / 3000
        return score
    best_slot = min(slots, key=preference_score)
    return best_slot["raw"]


```

```python
# 4) Full system function
1 usage
def process_patient_case(
    model,
    age: int,
    symptoms: List[str],
    pain_level: int,
    condition_type: str,
    vitals: Dict[str, str],
    preferred_date: str,
    preferred_time: str,
    available_slots: List[str]
) -> Dict[str, Any]:
    urgency_score = calculate_urgency_score(
        model=model,
        age=age,
        symptoms=symptoms,
        pain_level=pain_level,
        condition_type=condition_type,
        vitals=vitals
    )
    priority_level = classify_priority(urgency_score)
    checked_slots = check_doctor_availability(available_slots)
    recommended_slot = recommend_slot(
        priority_level=priority_level,
        preferred_date=preferred_date,
        preferred_time=preferred_time,
        available_slots=available_slots
    )
    if priority_level == "High":
        reason = "Urgent symptoms and/or abnormal vitals require earlier appointment."
    elif priority_level == "Medium":
        reason = "Moderate symptoms indicate a medium-priority appointment."
    else:
        reason = "Mild symptoms and stable vitals indicate low priority."
    return {
        "urgency_score": urgency_score,
        "priority_level": priority_level,
        "available_slots_checked": checked_slots,
        "recommended_slot": recommended_slot,
        "reason": reason
    }
# 5) Prototype test
if __name__ == "__main__":
    model = train_model()
    patient_result = process_patient_case(
        model=model,
        age=45,
        symptoms=["fever", "cough", "fatigue"],
        pain_level=5,
        condition_type="acute",
        vitals={"temperature": "high", "heart_rate": "normal"},
        preferred_date="tomorrow",
        preferred_time="afternoon",
        available_slots=["Today 2:00 PM - Dr. Ahmed","Tomorrow 1:00 PM - Dr. Sara","Tomorrow 3:00 PM - Dr. Ahmed"]
    )
    print(json.dumps(patient_result, indent=2))
```

Phase 4

AI feature 2: No show prediction

Feature Description:

This feature predicts the probability that a specific patient will miss their upcoming appointment by analysing historical data and booking context. Based on the predicted risk level, the system automatically generates an SMS reminder message, high-risk patients receive a double conformation message with also a overbook for their time slot, while low-risk patients receive only a reminder message.

This will directly supports reducing no-show rates by at least 40% as stated in the project objectives.

Processing logic:

Patient History> No-Show Prediction> recommended action (Appointment Notifications, Overbook)

System prompt:

You are a healthcare AI assistant, and your goal is predicting the no-show Risk.

Predict the probability from 0% to 100% that a patient will miss their next appointment. Given these factors: age, gender, total_past_appointments, no_show _history, appointment_type. you must give me no-show probability percentage also depending on the percentage identify the risk (low, medium, high) and recommend action (Reminder only, Double confirmation, Overbook), for the respond use JSON format.

Use few-shot learning technique

Some examples:

Example 1: Input: total_past_appointments=10, no_show_count=1, age=20, gender=”Male”, appointment_type=”chest pain”

Output: probability=12%, risk=Low, action="Send reminder only "

Example 2: Input: past_appointments=8, no_show_count=5, age=25, gender=”Male”, appointment_type=”Diabetes follow-up”

Output: probability=68%, risk=High, action=" send reminder message +Send double confirmation + do overbooking"

Example 3: Input: past_appointments=4, no_show_count=1, age=42, gender=”Female”, appointment_type=”General check-up ”

Output: probability=42%, risk=Medium, action="Send reminder message + send double confirmation "

Input:

Patient_id:603

age:35

gender:” Female ”

total_past_appointments:5

no_show_count:2

appointment_type: ”Routine Check-up”

Output:

{

"no_show_probability": 17.71%,

"risk_level": "Low",

"recommended_action": "Send reminder only " }

Python Code using Google Collab:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score
import json
import numpy as np

# For demonstration, let's create some dummy historical data
np.random.seed(42)
num_samples = 1000
age = np.random.randint(18, 80, num_samples)
gender = np.random.choice(['Male', 'Female'], num_samples)
total_past_appointments = np.random.randint(0, 20, num_samples)
appointment_type = np.random.choice(['Routine Check-up', 'Specialist Consult', 'Follow-up', 'Emergency'], num_samples)

# Ensure no_show_history is always <= total_past_appointments
# If total_past_appointments is 0, no_show_history must be 0
no_show_history = np.array([np.random.randint(0, tpa + 1) for tpa in total_past_appointments])

no_show_actual = np.random.choice([0, 1], num_samples, p=[0.8, 0.2]) # 0 for showed up, 1 for no-show

data = {
    'age': age,
    'gender': gender,
    'total_past_appointments': total_past_appointments,
    'no_show_history': no_show_history,
    'appointment_type': appointment_type,
    'no_show_actual': no_show_actual
}
df_historical = pd.DataFrame(data)

# Calculate no_show_rate from historical data
# Handle division by zero explicitly: if total_past_appointments is 0, no_show_rate is 0
df_historical['no_show_rate'] = np.where(
    df_historical['total_past_appointments'] > 0,
    df_historical['no_show_history'] / df_historical['total_past_appointments'],
    0
)

display(df_historical.head())


features = ['age', 'gender', 'total_past_appointments', 'no_show_rate', 'appointment_type']
target = 'no_show_actual'

X = df_historical[features]
y = df_historical[target]
```

```
y = df_historical[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Define categorical and numerical features
categorical_features = ['gender', 'appointment_type']
numerical_features = ['age', 'total_past_appointments', 'no_show_rate']

# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Create the full pipeline
model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(solver='liblinear', random_state=42))
])

# Train the model
model_pipeline.fit(X_train, y_train)

print("Model training complete.")


y_pred_proba = model_pipeline.predict_proba(X_test)[:, 1]
y_pred = model_pipeline.predict(X_test)

print(f"AUC Score: {roc_auc_score(y_test, y_pred_proba):.4f}")
print(f"Accuracy Score: {accuracy_score(y_test, y_pred):.4f}")



def predict_no_show_risk_ml(age: int, gender: str, total_past_appointments: int, no_show_history: int, appointment_type: str, ml_model: Pipeline) -> str:
    """
    Predicts the no-show probability, risk level, and recommends an action using a trained ML model.

    Args:
        age (int): The patient's age.
        gender (str): The patient's gender ('Male', 'Female', 'Other').
        total_past_appointments (int): Total number of past appointments the patient has had.
        no_show_history (int): Number of past appointments the patient has missed.
        appointment_type (str): Type of the appointment (e.g., 'Routine Check-up', 'Specialist Consult').
        ml_model (Pipeline): The trained scikit-learn pipeline for prediction.

    Returns:
```

```
    Returns:
        str: A JSON string containing the no-show probability, risk level, and recommended action.
    """

    # Prepare the input for the ML model
    input_data = pd.DataFrame([
        {
            'age': age,
            'gender': gender,
            'total_past_appointments': total_past_appointments,
            'no_show_rate': no_show_history / total_past_appointments if total_past_appointments > 0 else 0,
            'appointment_type': appointment_type
        }
    ])

    # Get probability from the ML model
    # The model predicts the probability of the positive class (no-show=1)
    no_show_probability = ml_model.predict_proba(input_data)[:, 1][0] * 100

    # Ensure probability is within 0-100
    probability = max(0, min(100, no_show_probability))

    # Determine risk level and recommended action
    if probability <= 30:
        risk_level = 'Low'
        recommended_action = 'Send reminder only'
    elif 31 <= probability <= 60:
        risk_level = 'Medium'
        recommended_action = 'Send reminder message + send double confirmation'
    else:
        risk_level = 'High'
        recommended_action = 'Send reminder message + send double confirmation + do overbooking'

    result = {
        "no_show_probability_percentage": f"{probability:.2f}%",
        "risk_level": risk_level,
        "recommended_action": recommended_action
    }

    return json.dumps(result, indent=2)


# Test with an example
print(predict_no_show_risk_ml(
    age=35,
    gender='Female',
    total_past_appointments=5,
    no_show_history=2,
    appointment_type='Routine Check-up',
    ml_model=model_pipeline
```

```
    ml_model=model_pipeline
))

print(predict_no_show_risk_ml(
    age=70,
    gender='Male',
    total_past_appointments=10,
    no_show_history=0,
    appointment_type='Follow-up',
    ml_model=model_pipeline
))

print(predict_no_show_risk_ml(
    age=25,
    gender='Male',
    total_past_appointments=1,
    no_show_history=1,
    appointment_type='Specialist Consult',
    ml_model=model_pipeline
))
```

Output:

	age	gender	total_past_appointments	no_show_history	appointment_type	no_show_actual	no_show_rate
0	56	Male	14	7	Follow-up	0	0.500000
1	69	Male	6	4	Follow-up	0	0.666667
2	46	Female	8	0	Follow-up	1	0.000000
3	32	Male	18	8	Emergency	0	0.444444
4	60	Male	15	0	Follow-up	0	0.000000

```
Model training complete.
AUC Score: 0.4729
Accuracy Score: 0.8100
{
  "no_show_probability_percentage": "19.57%",
  "risk_level": "Low",
```

	age	gender	total_past_appointments	no_show_history	appointment_type	no_show_actual	no_show_rate
0	56	Male	14	7	Follow-up	0	0.500000
1	69	Male	6	4	Follow-up	0	0.666667
2	46	Female	8	0	Follow-up	1	0.000000
3	32	Male	18	8	Emergency	0	0.444444
4	60	Male	15	0	Follow-up	0	0.000000

```
Model training complete.
AUC Score: 0.4729
Accuracy Score: 0.8100
{
  "no_show_probability_percentage": "19.57%",
  "risk_level": "Low",
```